

UFO Witness Report Analyzer

What the project is and how it works

Sammuel Hinton · ICT 890-015, Machine Learning & AI Bootcamp · University of Calgary Continuing Education (delivered by RoboGarden)

What it is

You paste in a UFO sighting report, the kind of thing someone types online, and the model guesses the shape of what they saw. Triangle, disk, fireball, light, that sort of thing. It also shows how confident it is and pulls out a few keywords from the report.

It's a text classifier built on TF-IDF and Naive Bayes, trained on about eighty thousand real reports. It's not an LLM, and it doesn't prove anybody saw anything. It just sorts text into shape labels. The friendly name is the Witness Report Analyzer; the technical name is a multi-class shape classifier.

The problem

Each report gives me two things I care about. The witness's written description, which is what the model reads, and the shape they picked, which is the answer I want it to learn. So it's supervised classification: show the model thousands of labeled examples, then let it guess the shape of a report it's never seen.

I went with this instead of the usual spam or movie-review project because the text is real and messy, there are eight shapes instead of a simple yes or no, and one shape completely dominates the data, which forces you to be honest about which numbers actually mean anything.

The data

It's a public, cleaned-up set of NUFORC reports sitting on GitHub, so the notebook just pulls it off a link, no login. The raw file is around eighty thousand rows, and annoyingly it has no header row, so if you don't name the columns yourself pandas grabs your first real report and treats it as the titles.

After cleaning and keeping the eight shapes that show up often enough to learn from (light, triangle, circle, fireball, sphere, disk, oval, cigar), I'm left with about fifty-four thousand reports. I drop "unknown" since it isn't really a shape. The big thing to know is that "light" swamps everything else, and that colors every decision after it.

How it works

First the text gets cleaned up: decode the HTML junk in the comments, tidy the spacing, lowercase the shape labels so “Triangle” and “triangle” count as one, and toss any report too short to say anything.

Then TF-IDF turns each report into numbers, because the model can't read English the way we do. It scores a word higher when it actually matters in a report and near zero for filler like “the” that shows up everywhere. I let it use two-word phrases too, so something like “three lights” can count as its own thing. Think of it like a toolbox: common bolts everybody has tell you nothing, but one specialty tool tells you the job. TF-IDF hunts down the specialty words.

Naive Bayes does the deciding, and it's basically a detective stacking up small clues. In training it learns which words lean toward which shape, and on a new report it weighs the words one by one and picks the best bet. It's fast, it's a solid baseline for text, and I can explain every piece of it, which is the whole point for a capstone. TF-IDF and Naive Bayes are wired together in one pipeline, and the train/test split is stratified 80/20 with a fixed seed so the rare shapes stay balanced and the results repeat.

How well it does

Because “light” is such a huge chunk of the data, accuracy on its own lies to you. A lazy model could just guess “light” a lot and still look okay. So I look at a few numbers together and lean on macro-F1 and the confusion matrix.

Metric	Score	What it tells me
Accuracy	0.57	How many test reports it got right overall.
Macro-F1	0.51	Treats all eight shapes equally. The number I trust most.
Weighted-F1	0.54	Same idea, but leans toward the common shapes.

The confusion matrix says the real story. Shapes with their own distinctive words, like triangle, fireball and cigar, do fine. The round ones (circle, sphere, oval) get mixed up because people describe them all the same way, with “round” and “glowing” and “ball.” The model can't tell apart what our own language doesn't, and honestly that was the most useful thing I learned. It's a data problem, not a code bug.

The analyzer bits

On top of the plain classifier I added the stuff that makes it feel like an actual tool. The predicted shape and its confidence come straight from the model. The keywords are just the highest-scoring words in your report, so you can see what tipped it. And there's a stretch feature that finds a few of the most similar old reports using cosine similarity, with their location and year.

Stuff that broke

A few things tripped me up, and they were the good kind of learning.

- The file had no header row, so my first load used a real report as the column names and everything looked cursed until I named them myself.
- There was HTML junk in the text, like a code that just means a comma, that I had to decode or the words came out weird.
- Everything wanted to be “light,” because that's most of the training data. Accuracy looked better than the model really was until macro-F1 and the confusion matrix set me straight.
- “Three lights” isn't the same as triangle to this model. It reads words, not shapes, so if nobody writes “triangle” it tends to fall back to light. That's just how these models work.

Limits

The comments are short and the labels are noisy, so there's a real ceiling here no matter what. “Light” soaks up a lot of the vague cases. It's a classic baseline, not a big language model. And to be clear, it doesn't tell you if any sighting is real. It sorts text labels in a public dataset, and every slide and demo says exactly that.

The bonus: the website

Last thing, and it's the fun one. The notebook is mine. The website's code, the HTML and JavaScript, I built with an AI assistant. I kept it because the trick behind it is too cool not to show.

It runs the model with no server. Normally a site like this ships your text off to some computer that does the thinking and sends back an answer. This one doesn't. It all happens right in your browser, which is why it can sit for free as plain files, and it even works offline.

A model, stripped down, is just a pile of numbers plus some math. When training finished, those numbers got dropped straight into the page, and the scoring math got rewritten in JavaScript so the browser can do it. It's like shipping the finished blueprint instead of the whole factory: the page carries the blueprint, and running a model right on your device like that has a real name, on-device inference, that the big companies use too.

The one honest wrinkle: the example buttons show the exact numbers my Python model gave, since I saved those. Anything you type fresh runs on the browser version, which gets really close to Python but won't always match to the last decimal. Fine for a demo, just not the identical model.

In short

It's the full run: load real data, clean it, look at it, split it, turn text into numbers with TF-IDF, train Naive Bayes, and check it carefully because the classes are so lopsided, then wrap it in an analyzer that gives you a shape, a confidence, and some keywords. The machine learning is mine. The website's a bonus front-end with a genuinely useful trick behind it.

Instructors: Omar Ashinawy, Hesam Damghanian, and Amber Carney. With thanks to Mostafa Moharram for the extra help.

Sammuel Hinton

(780) 838-7496 · samhinton310@gmail.com